

次元の呪いと次元削減

特徴量を増やし続けると何が起きるか

rkskmt

1

ここまでの流れ

- **重さだけ**でサーモンを分類 → ある程度できた
- **輝度も追加**して2次元で分類 → 精度が上がった
- ならば**特徴量を増やし続ければ**もっと精度が上がる？

① 今回の問い

特徴量（次元）を増やし続けると何が起きるか？

2

次元の呪い

3

次元の呪い（Curse of Dimensionality）

“データの次元が増えるにつれて、空間が指数的に広くなり、データが希薄になる”
—— Richard Bellman（1961）

- 2次元で十分だったサンプル数が、10次元では**全く足りなくなる**
- 高次元空間は私たちの直感とは全く異なる振る舞いをする

① ノート

1960年代にベルマン（動的計画法の発明者）が提唱した概念。 現代機械学習でも最重要トピックの一つ。

4

クイズ：いちばん近い点まで、どれくらい？

- 各特徴量は 0～1 に正規化してあるとする（各軸の長さ＝1）
- そこに**100個の点**をランダムにばらまく
- **1次元**なら、いちばん近い点までの距離はだいたい **0.003**（ぎゅうぎゅう）

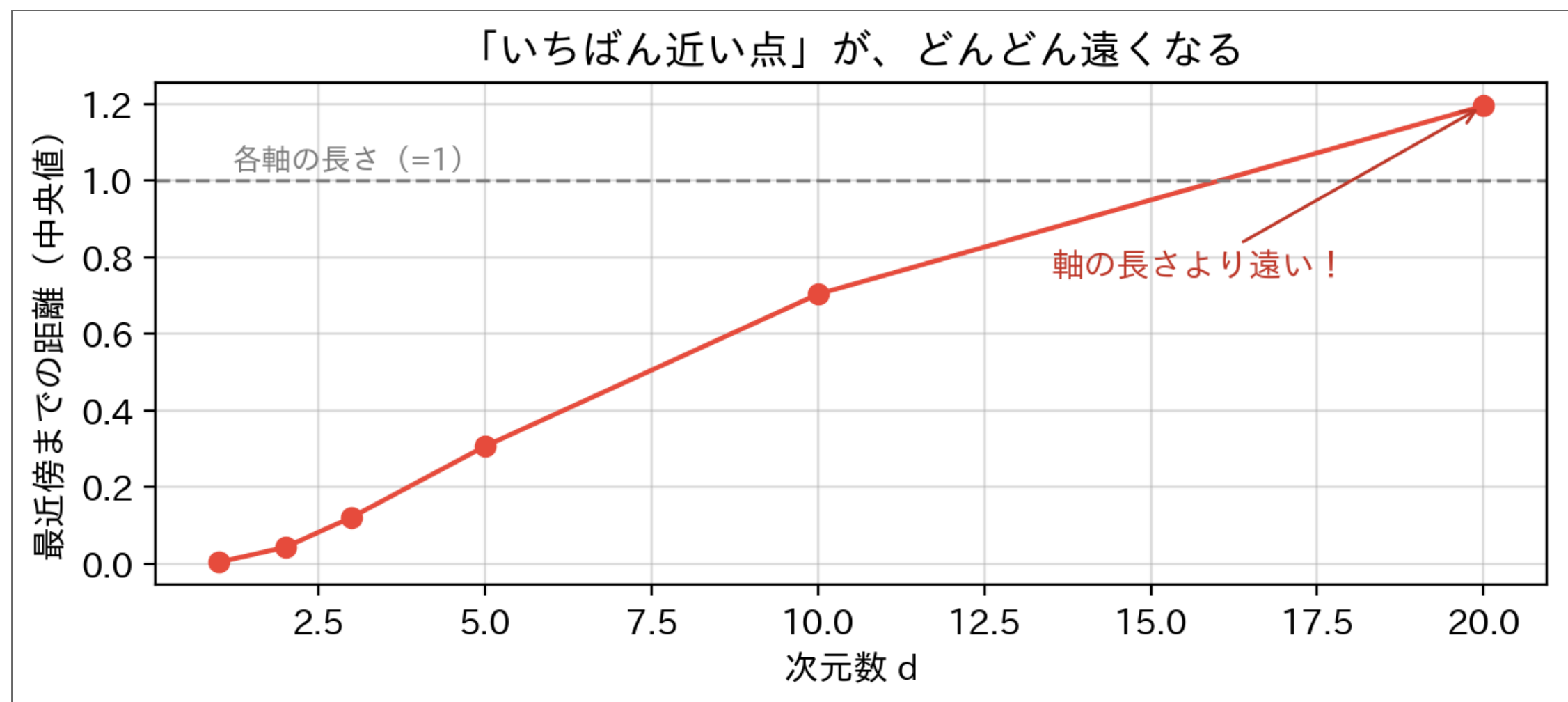
❗ クイズ

特徴量を増やして **10次元** にしたら、いちばん近い点までの距離はいくつになると思う？

（各軸の長さは1のまま。点の数も100個のまま）

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4 plt.rcParams['font.size'] = 11
5 plt.rcParams['figure.dpi'] = 100
6
7 rng = np.random.default_rng(0)
8 dims = [1, 2, 3, 5, 10, 20]
9 nn_median = []
10
11 for d in dims:
12     P = rng.random((100, d))          # 100点を [0,1]^d にばらまく
13     D = np.sqrt(((P[:, None, :] - P[None, :, :]) ** 2).sum(axis=2))
14     np.fill_diagonal(D, np.inf)        # 自分自身は除外
15     nn_median.append(np.median(D.min(axis=1))) # 最近傍距離の中央値
16
17 for d, m in zip(dims, nn_median):
18     print(f"{d:2d}次元: いちばん近い点まで {m:.3f}")
```

1次元: いちばん近い点まで 0.003
2次元: いちばん近い点まで 0.042
3次元: いちばん近い点まで 0.121
5次元: いちばん近い点まで 0.306
10次元: いちばん近い点まで 0.703
20次元: いちばん近い点まで 1.195



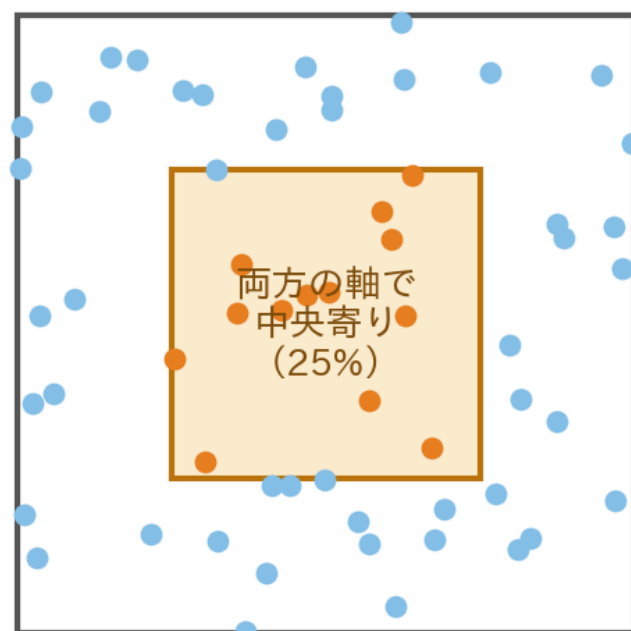
- 10次元で **0.67**、20次元では **1.2** — もはや「お隣さん」は存在しない
- 同じ100点なのに。なぜこんなことが起きるのか、2つの理由を見ます

なぜ？ その1：真ん中に居続けられない

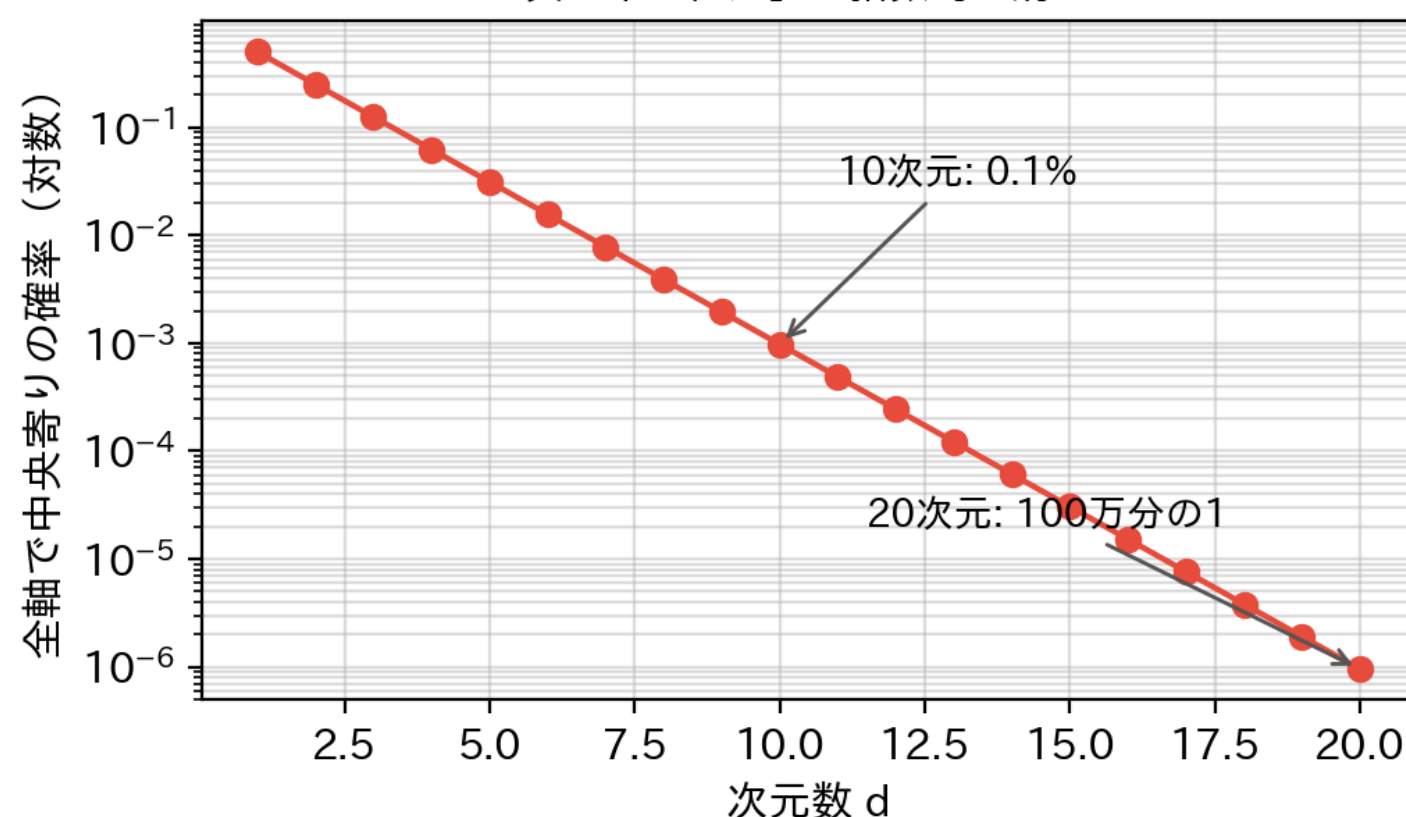
- ある点が、ある軸で「中央寄り」（その軸の **中央50%**）に入る確率は $\frac{1}{2}$
- d 次元で「**全部の軸**で中央寄り」でいられる確率は：

$$\left(\frac{1}{2}\right)^d \quad \text{1次元: 50\%} \quad \text{3次元: 12.5\%} \quad \text{10次元: 0.1\%} \quad \text{20次元: 100万分の1}$$

2次元：もう4人に1人しか残れない



「真ん中の住人」は指数的に消える



- ほぼすべての点が、**どこかの軸では端っこ** — つまり「角のあたり」に住むことになる
- しかも角の数は 2^d 個（20次元で約105万個）。**データは100万個の角に散り散り** になる

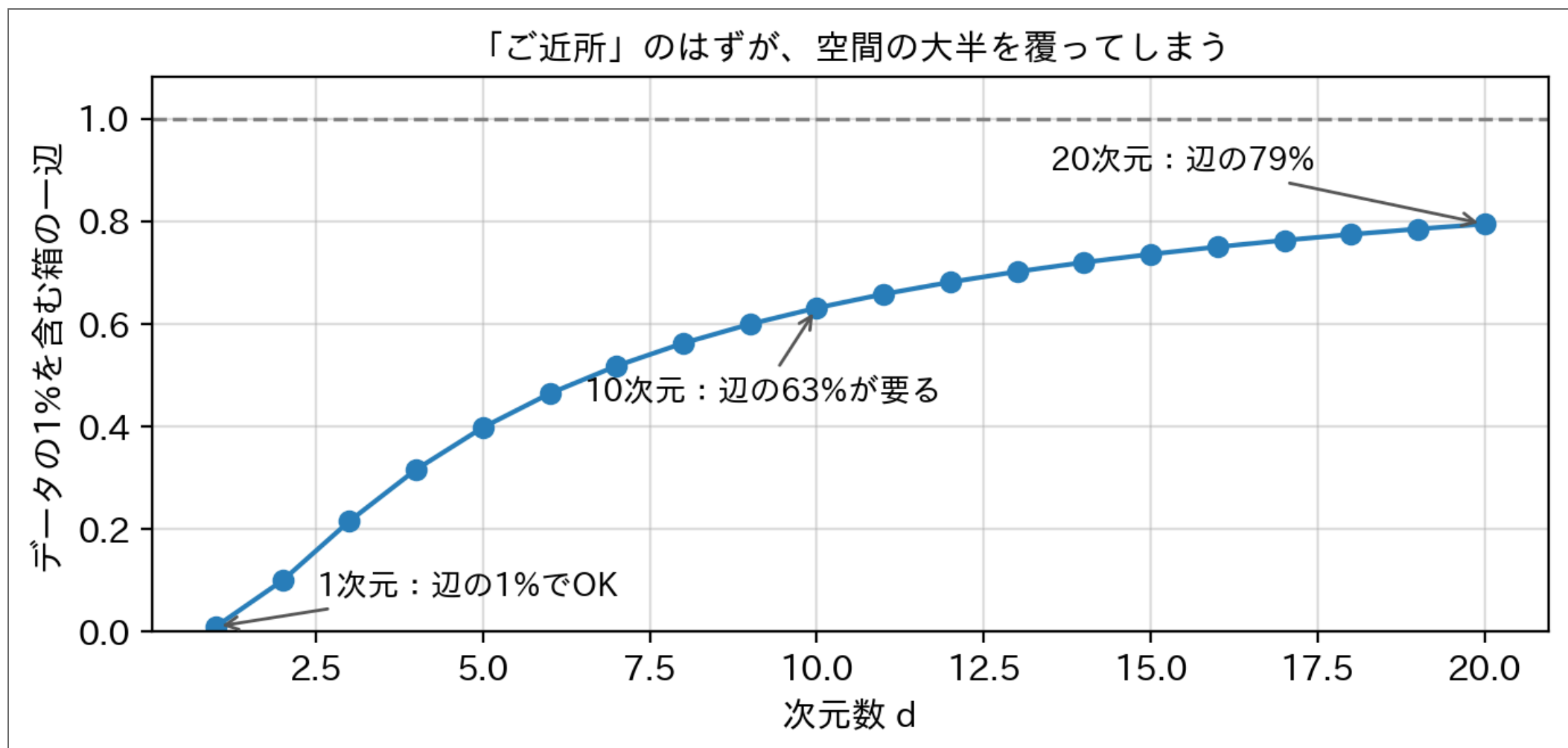
① ノート

教科書によく出る言い換え：「単位立方体に内接する球（＝どの軸でも中央寄りの領域）の体積は、高次元で 0 に向かう」。いま見たのと同じ現象。

なぜ？ その2：「ご近所」を集めると世界を覆ってしまう

- k-NN の気持ちになって、周りの **データの1%** を「ご近所」として集めたい
- 各軸を割合 r だけ切り取った箱は、データの r^d 割を含む → 1% 集めるには：

$$r = 0.01^{1/d}$$

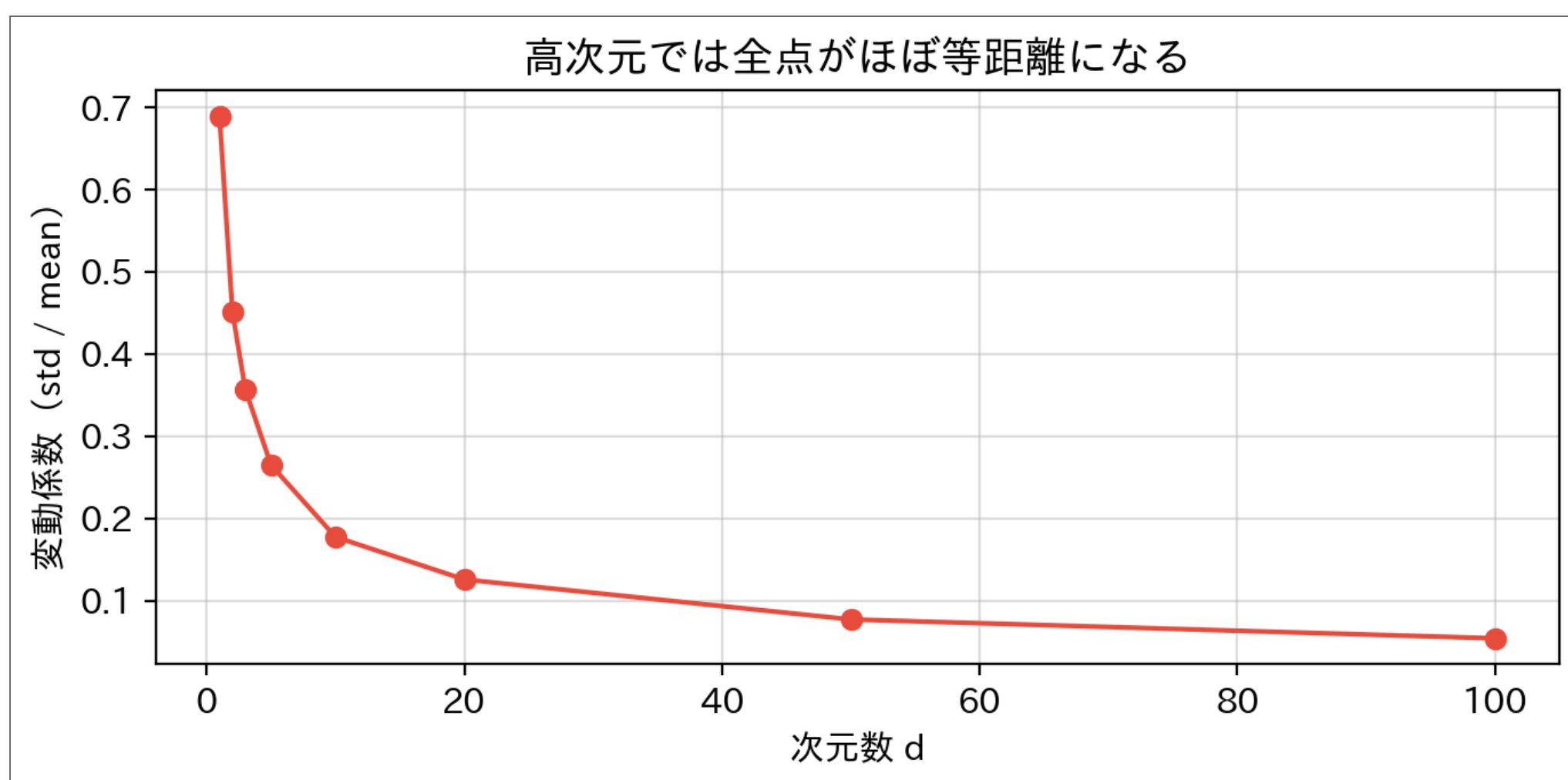


- 10次元では、たった1%の「ご近所」を集めるのに **各軸の63%** を見渡す必要がある
- それはもう「近所」ではない — **「近い」に頼る手法 (k-NN など) が最初に壊れる**

距離が意味をなさなくなる

- k-NN やロジスティック回帰など、多くの手法は**距離**に依存している
- 高次元では最近傍と最遠傍の距離差が消えていく

```
1 np.random.seed(42)
2 dims = [1, 2, 3, 5, 10, 20, 50, 100]
3 cv_list = [] # 変動係数 = std / mean
4
5 for d in dims:
6     pts = np.random.rand(2000, d)
7     query = np.random.rand(d)
8     dists = np.sqrt(((pts - query) ** 2).sum(axis=1))
9     cv_list.append(dists.std() / dists.mean())
10
11 plt.figure(figsize=(7, 3.5))
12 plt.plot(dims, cv_list, "o-", color="#e74c3c")
13 plt.xlabel("次元数 d")
14 plt.ylabel("変動係数 (std / mean) ")
15 plt.title("高次元では全点がほぼ等距離になる")
16 plt.grid(True, alpha=0.4)
17 plt.tight_layout()
18 plt.show()
```

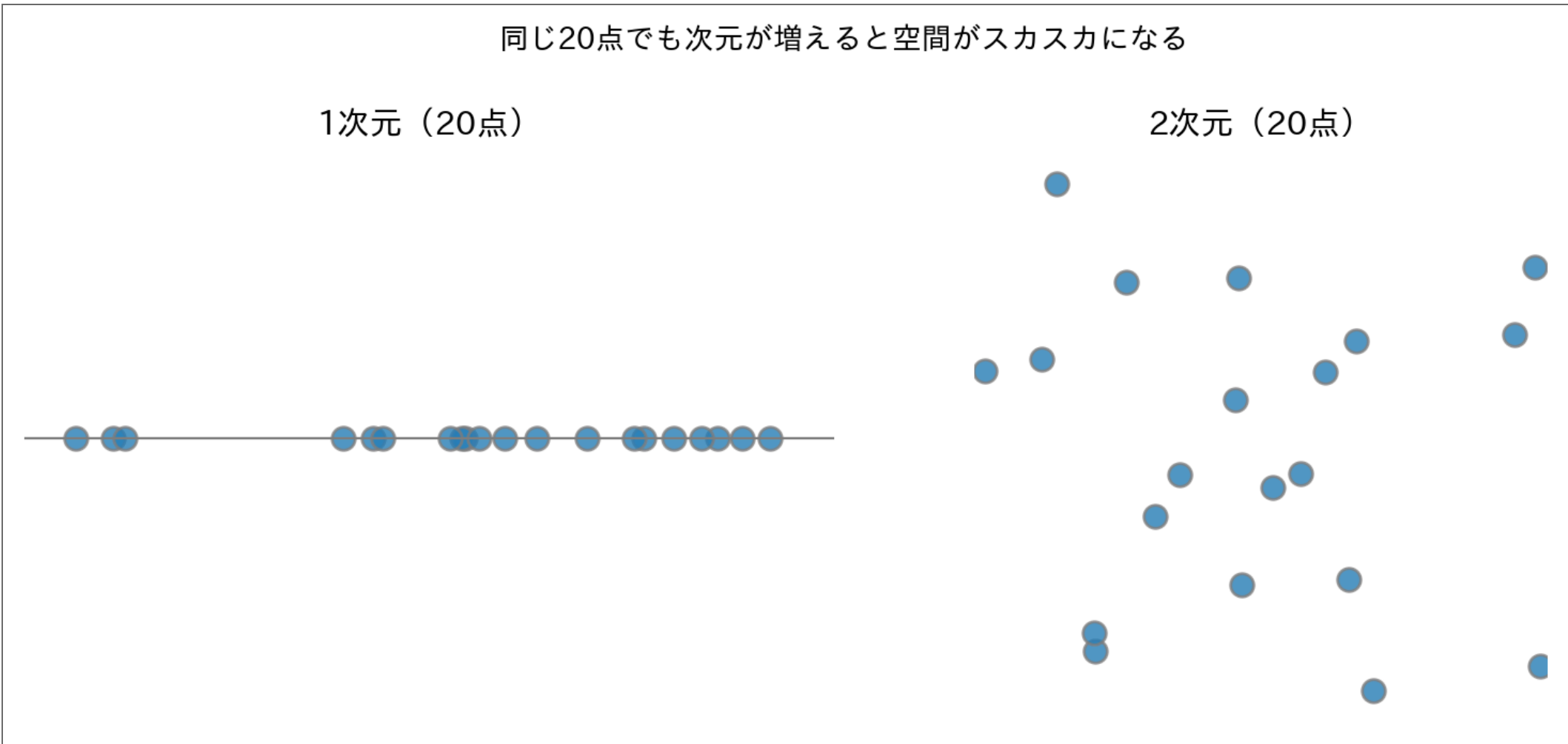


⚠ 変動係数が 0 に近づく

「最も近い点」と「最も遠い点」の距離がほぼ同じになる。「近い」という概念が意味をなさなくなる。

データが疎になる（Sparsity）

- 同じサンプル数でも、次元が増えると空間が広くなりすぎる



① ノート

密度を保つには、次元数に対して**指数的にサンプルを増やす**必要がある。10次元で十分な密度を確保するには、1次元の 10^{10} 倍のサンプルが必要という計算になる。

実用への影響

問題	原因
モデルの精度が上がらない	データが疎でパターンが見えない
過学習しやすくなる	データに比べてパラメータが多すぎる
計算コストが増大	距離計算などが次元数に比例して重くなる
可視化できない	3次元以上は直接見られない

💡 次元の呪いへの対処

1. **特徴量選択**：重要な特徴量だけを残す
2. **次元削減**：情報を保ったまま次元を圧縮する（← 今回）
3. **正則化（regularization）**：過学習を防ぐ（回帰問題で詳しく）

3つ目の特徴量：体長

- 重さ・輝度で精度が上がった。気を良くして、80匹全部の**体長**も測った
 - ▶ 魚データの定義（クリックして展開）

体長（先頭5匹）： [58.4 55.3 59.2 61.7 54.1] cm
体長の範囲： 45.0～61.7 cm

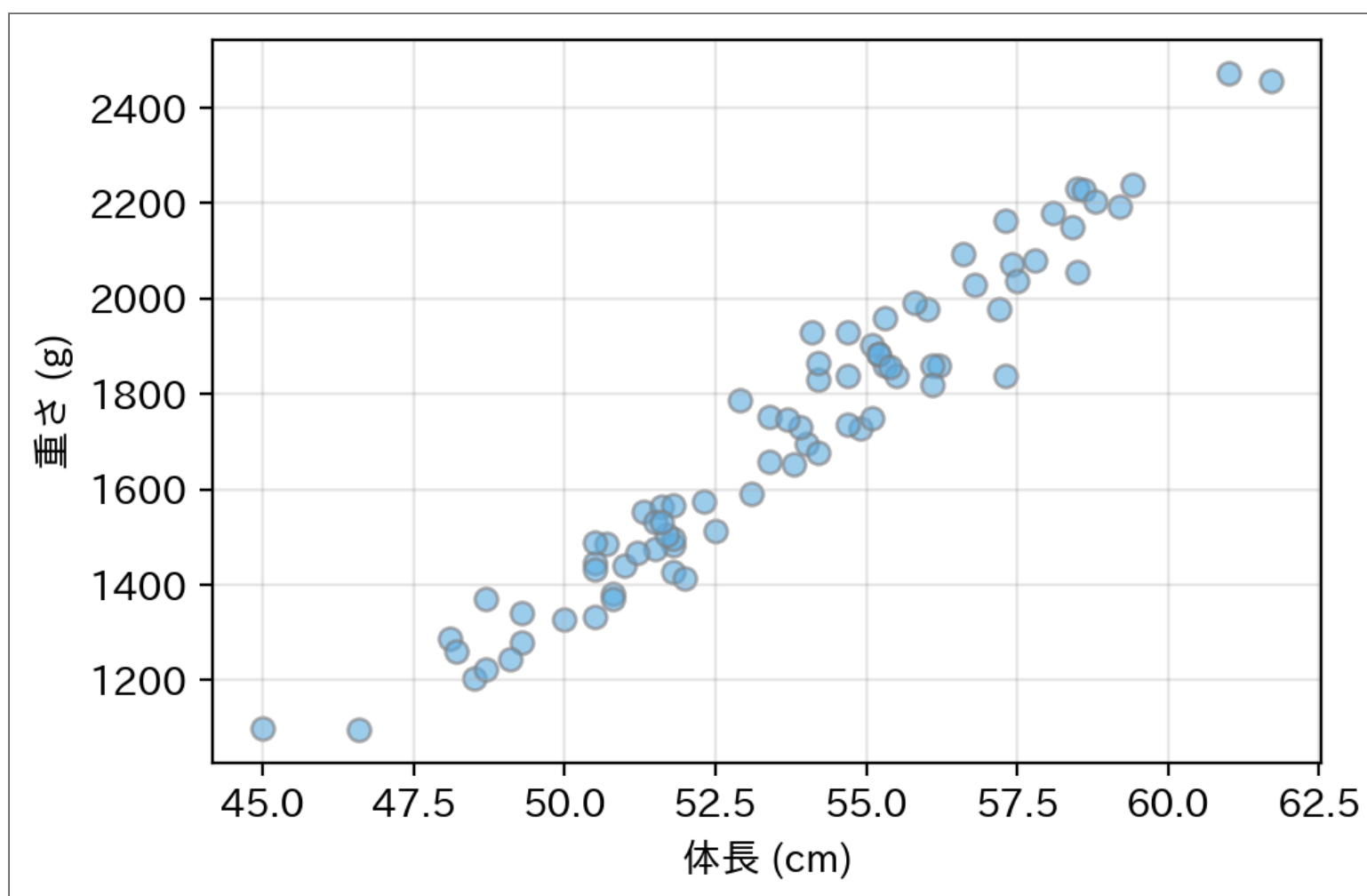
🚨 クイズ

特徴量が 2 → 3 に増えた。**手に入る情報も 1.5 倍**になったと思う？

12

体長と重さを並べてみる

```
1 plt.figure(figsize=(5.5, 3.6))
2 plt.scatter(lengths, weights, alpha=0.6, s=40, c="#5dade2", edgecolors="gray")
3 plt.xlabel("体長 (cm)"); plt.ylabel("重さ (g)")
4 plt.grid(True, alpha=0.3)
5 plt.tight_layout()
6 plt.show()
7
8 print(f"相関係数: {np.corrcoef(lengths, weights)[0, 1]:.3f}")
```



相関係数： 0.973

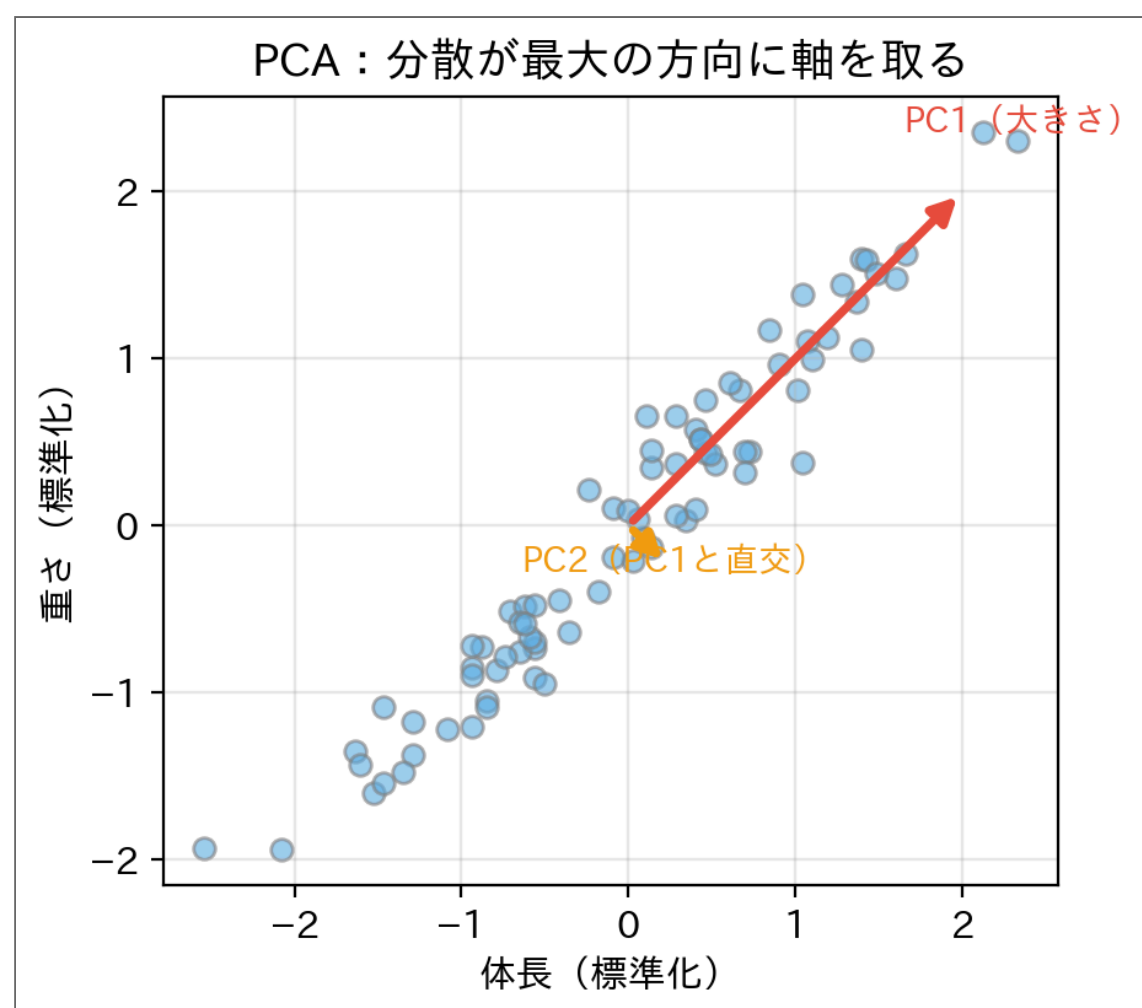
- ほぼ**一直線**。長い魚は重い — あたりまえだった
- 体長と重さは、**ほぼ同じ情報を2回測っていた**
- この「実は重複している」を自動で見つけて畳む道具が **PCA**

13

PCA の直感

Principal Component Analysis（主成分分析）

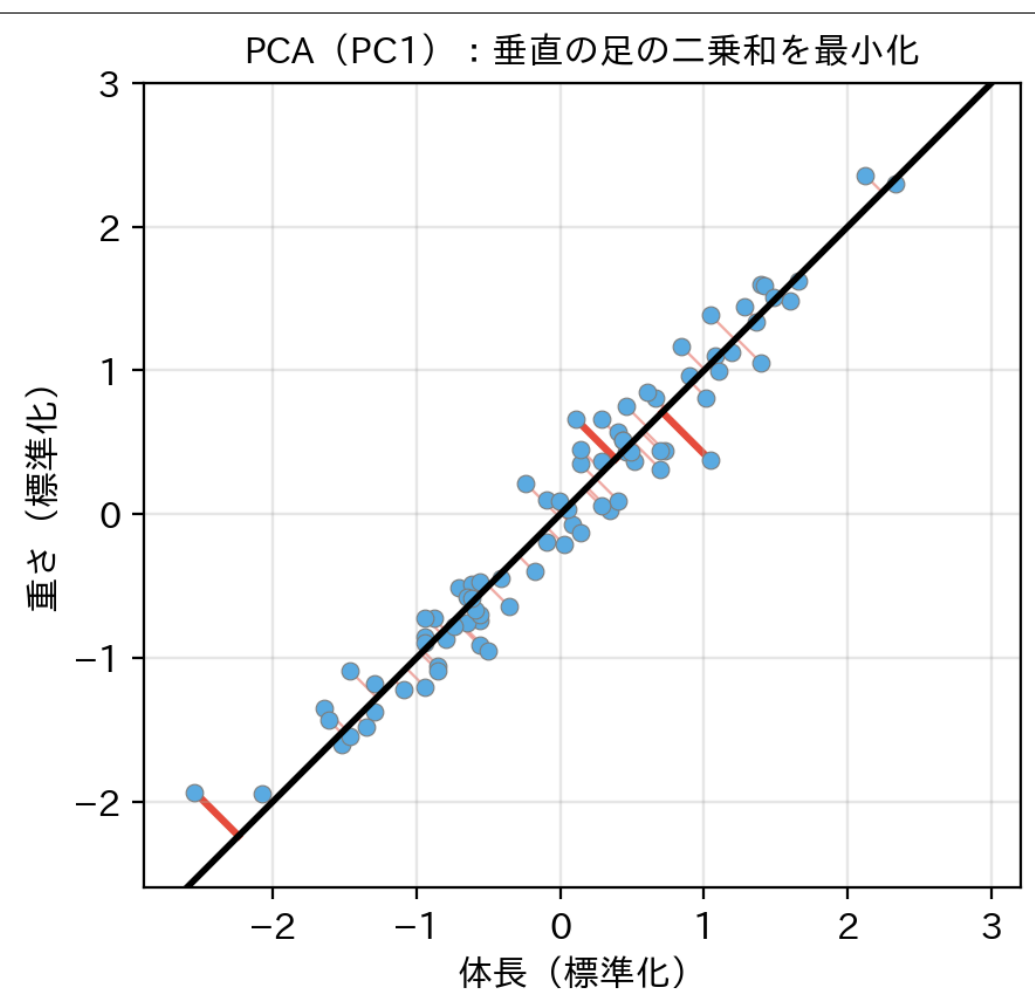
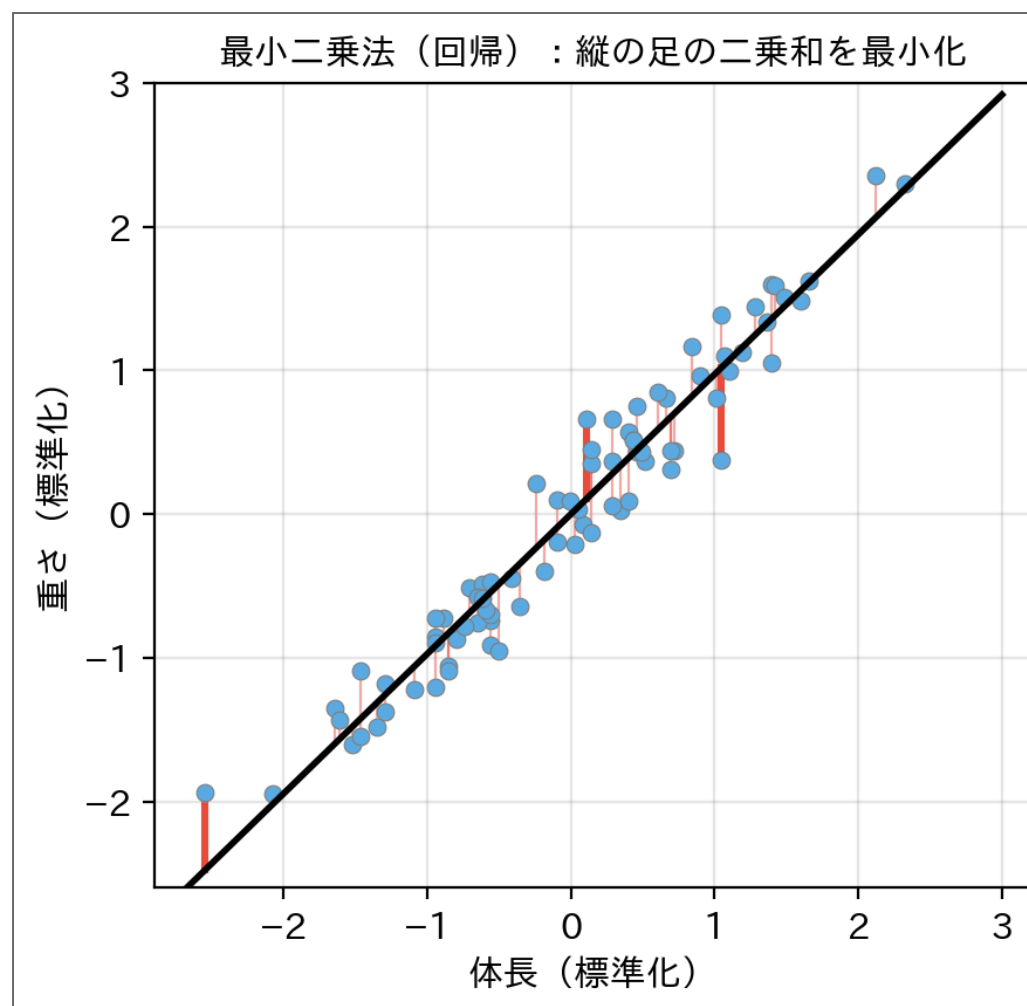
- 斜めに細長い雲は、もとの2軸より「**長い方向**と**細い方向**」で見るほうが自然
- データの**分散（variance）が最大になる方向**に新しい軸（主成分）を取る



- PC1 は体長と重さを合わせた「**大きさ**」の軸 — この方向だけで分散の約 99% を説明できる

最小二乗法と PCA — 「足」の下ろし方が違う

- 実はどちらも「**足（誤差）の二乗和が最小になる直線**」を探している
- 違うのは**足の向き**だけ



- **最小二乗法（線形回帰の回帰の MSE）**：仕事は「 y を当てる」→ 誤差は**縦**に測る
- **PCA**：仕事は「データの形を要約する」→ 誤差は直線への**最短距離（垂直の足）**で測る
- 足の向きが違うので、選ばれる直線も**少し違う**（回帰の傾き 0.97、PC1 の傾き 1.00）

① 「分散最大」と「垂直の足最小」は同じこと

各点で（中心からの距離） $^2 =$ （軸上の影） $^2 +$ （垂直の足） 2 （ピタゴラスの定理）。左辺の合計はデータで決まっていて動かないので、**影の分散を最大にする軸と 足の二乗和を最小にする直線**は、同じものを選ぶ。

「分散が最大の方向」を式にする

- 探している **直線の向き** を、 **単位ベクトル $\mathbf{v}$** ($\|\mathbf{v}\| = 1$) で表す
- 中心化した点 $\tilde{\mathbf{x}}_i$ のその直線への **影 (射影, projection)** は、内積 $z_i = \tilde{\mathbf{x}}_i^\top \mathbf{v}$ (ただの数)
- 80匹ぶんの影は、行列とベクトルの積で **一度に出せる** : $\mathbf{z} = \tilde{X}\mathbf{v}$

知りたい「影の分散」を、少しずつ変形します：

$$\begin{aligned}\text{影の分散} &= \frac{1}{n-1} \sum_i z_i^2 = \frac{1}{n-1} \mathbf{z}^\top \mathbf{z} && \text{(2乗和は「自分との内積」)} \\ &= \frac{1}{n-1} (\tilde{X}\mathbf{v})^\top (\tilde{X}\mathbf{v}) = \frac{1}{n-1} \mathbf{v}^\top \tilde{X}^\top \tilde{X} \mathbf{v} && \text{(転置の規則 } (AB)^\top = B^\top A^\top \text{)} \\ &= \mathbf{v}^\top \underbrace{\left(\frac{1}{n-1} \tilde{X}^\top \tilde{X} \right)}_C \mathbf{v} && \text{(真ん中のかたまりに名前をつける)} \\ &&& C \text{ (共分散行列)}\end{aligned}$$

- **共分散行列 (covariance matrix) C** は、**影の分散を変形すると自然に現れる** (ここが登場理由)
- つまり PCA の探しものはこう書ける：

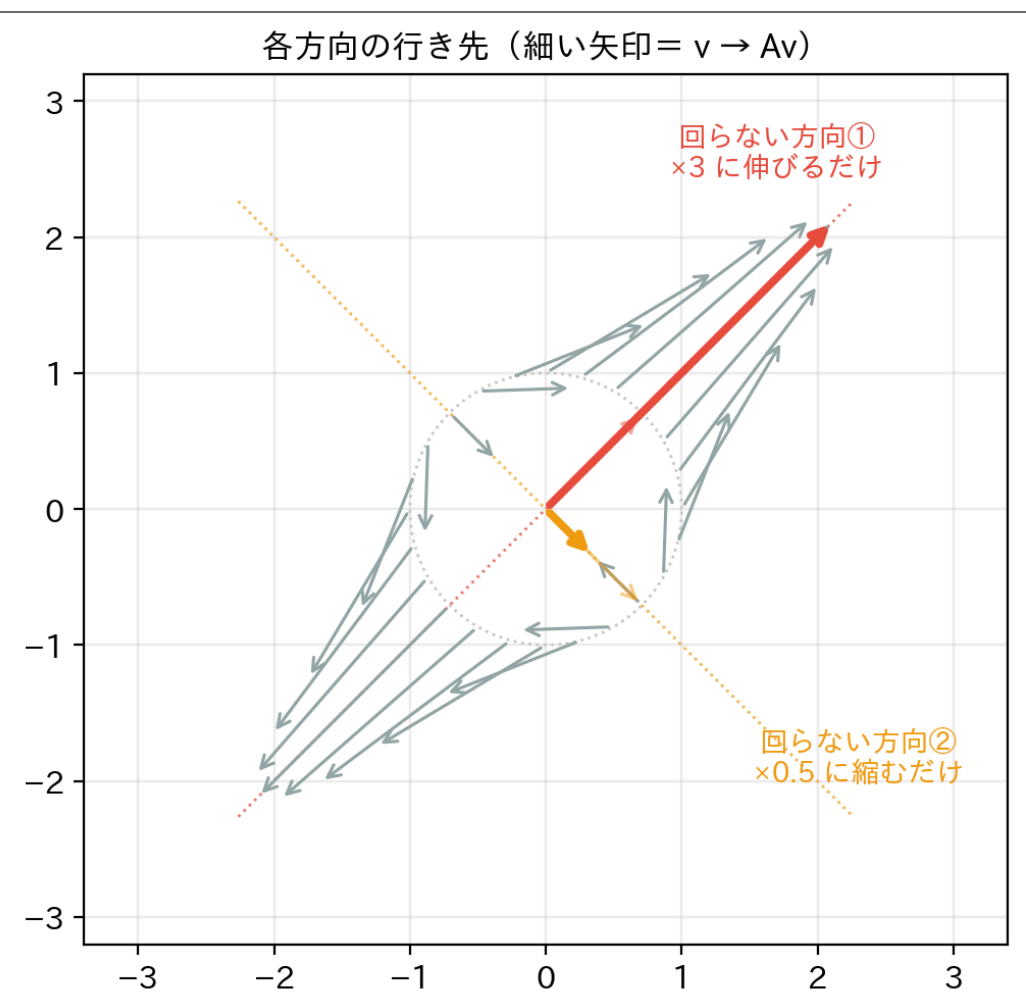
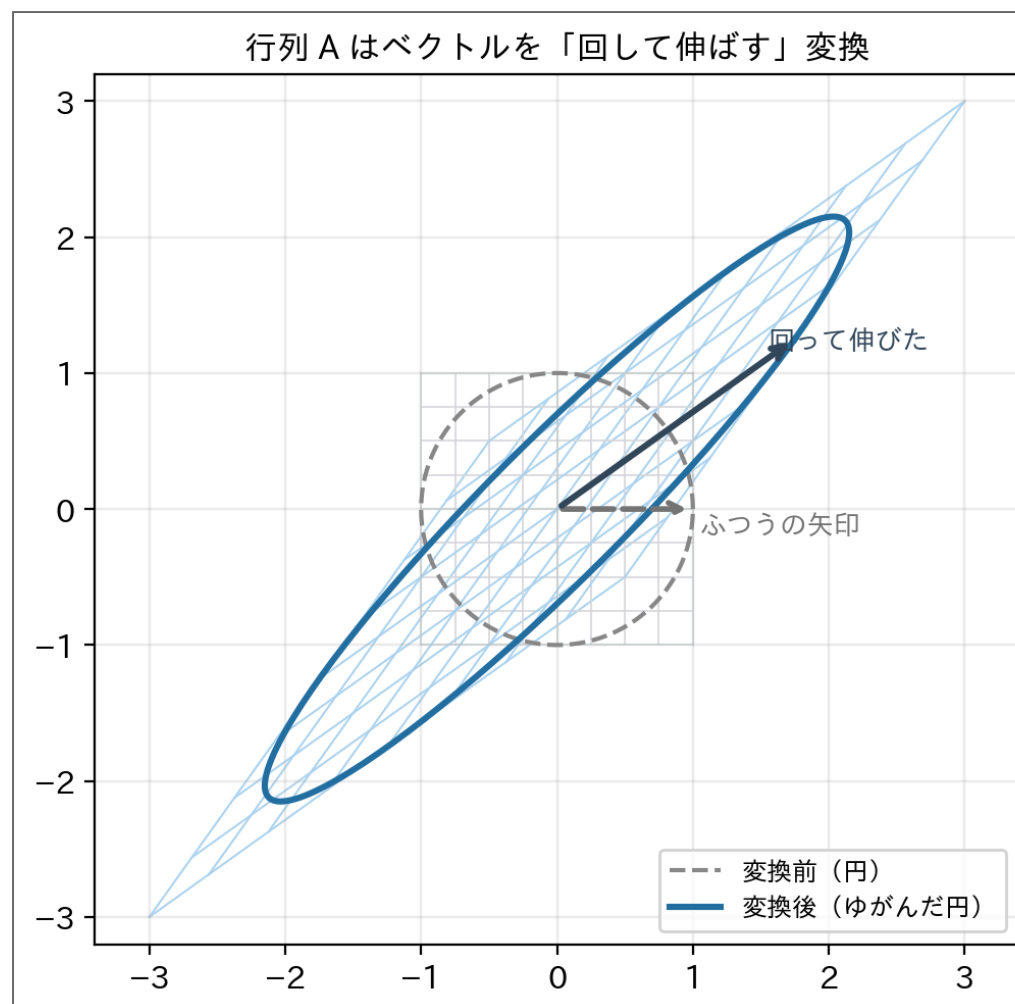
① ノート

$\|\mathbf{v}\| = 1$ のもとで $\mathbf{v}^\top C \mathbf{v}$ (= $\mathbf{v}$ 方向の分散) を **最大にする $\mathbf{v}$** を探せ

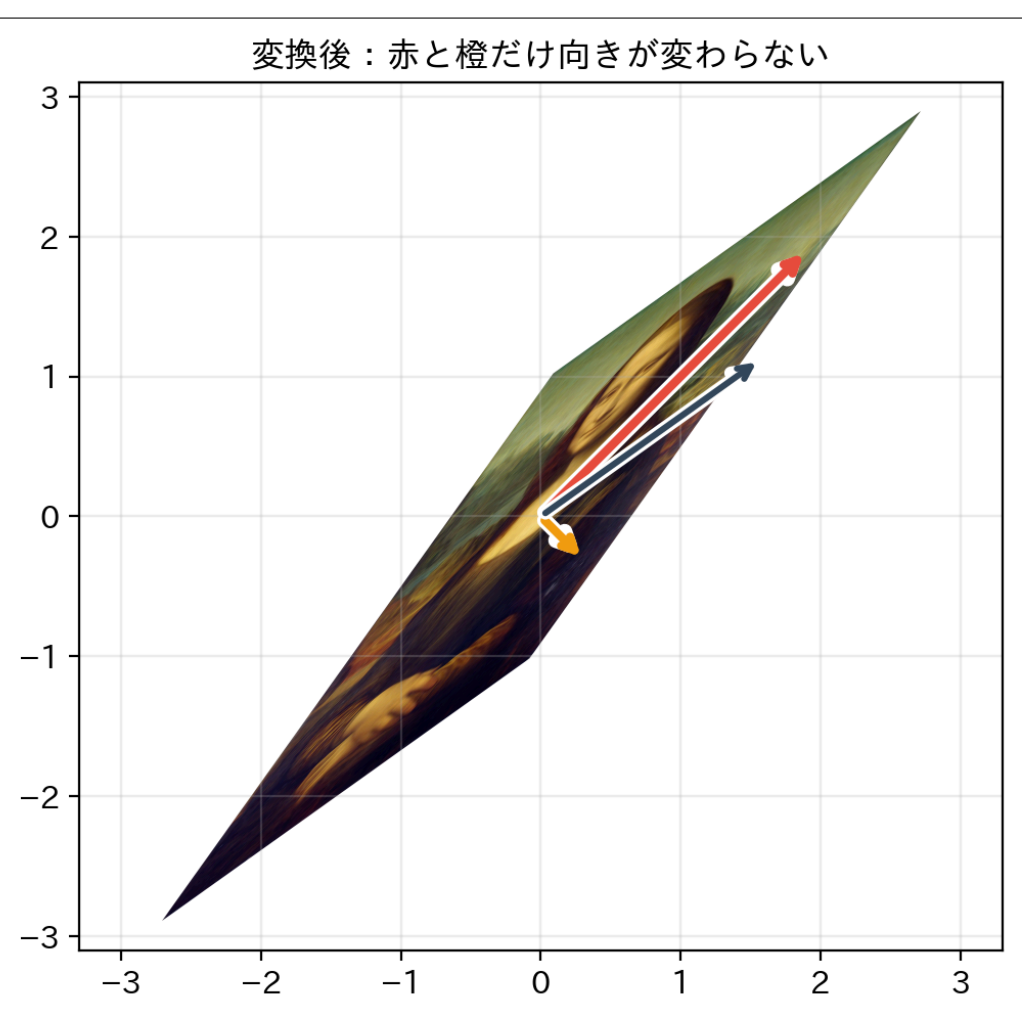
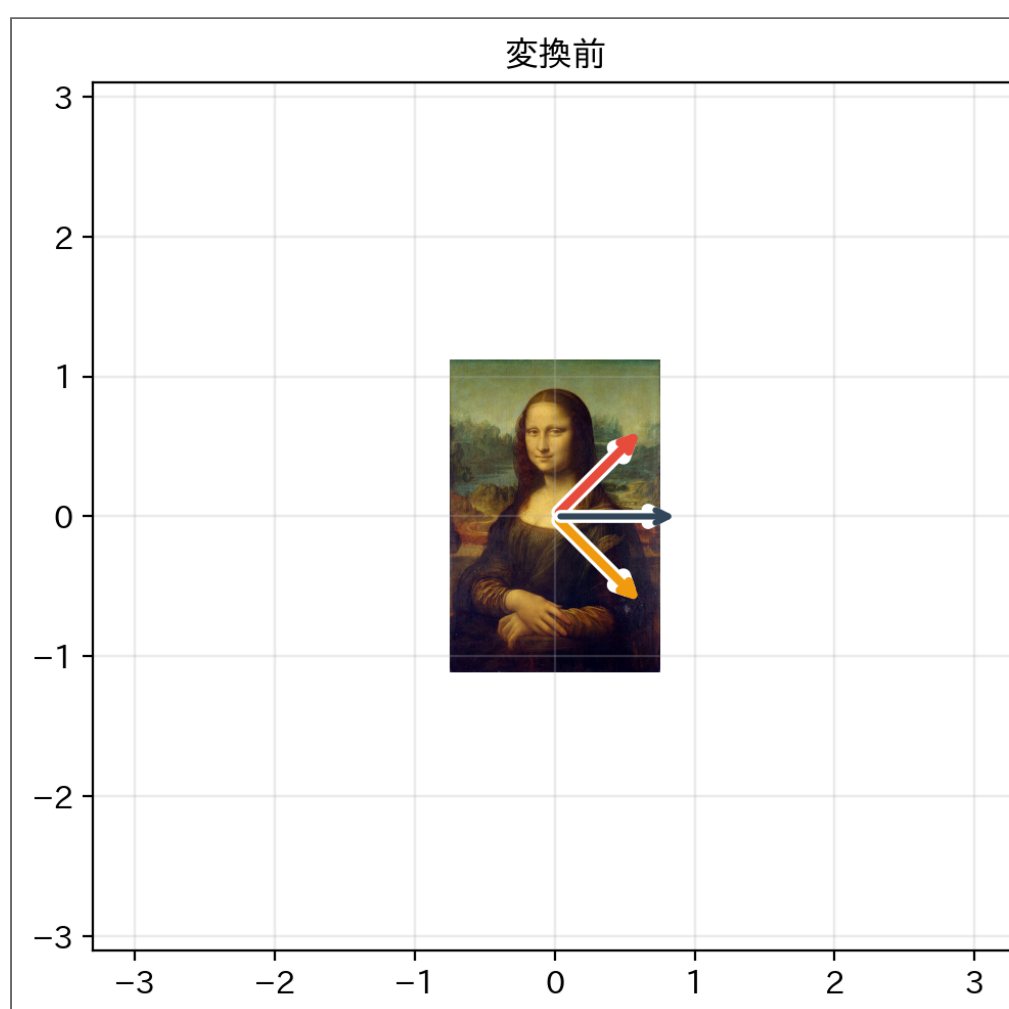
ところで「固有ベクトル」と「固有値」とは

- 行列 A は、ベクトルを別のベクトルへ移す **変換**（回して・伸ばす機械）。次の A で実験：

$$A = \begin{pmatrix} 1.75 & 1.25 \\ 1.25 & 1.75 \end{pmatrix}$$



同じ変換を、名画にかけてみると：



（絵：レオナルド・ダ・ヴィンチ 《モナ・リザ》 — [Wikimedia Commons](#)、パブリックドメイン）

- ほとんどの矢印は、変換されると **回ってしまう**（「ふつうの矢印」＝紺色）
- でも **回らない方向が2本だけ** がある（赤と橙） — 向きはそのまま、**伸び縮みするだけ**
- その方向が **固有ベクトル（eigenvector）** $\mathbf{v}$ 、伸び縮みの倍率が **固有値（eigenvalue）** λ ：
 $A\mathbf{v} = \lambda\mathbf{v}$ （向きはそのまま、長さだけ λ 倍）

① ノート

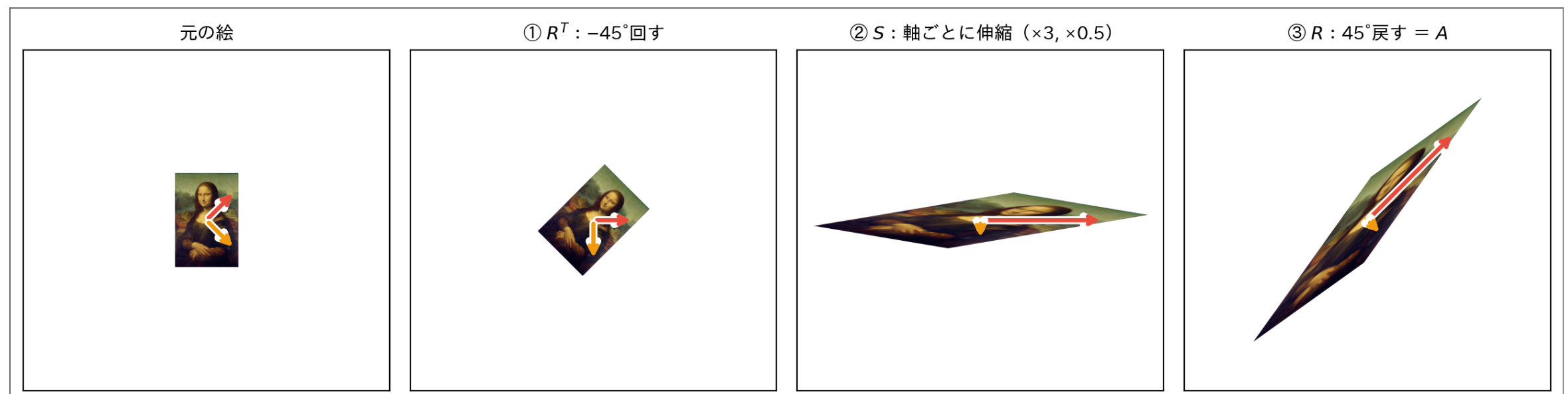
共分散行列 C も、これと同じ「対称な変換」のひとつ。このあと、 C の回らない方向こそが「分散最大の軸」だと分かる。

A の正体：回して・伸ばして・戻す

種明かし：この A は、おなじみの **回転行列** と **軸ごとの伸縮** だけで組み立てててありました。

$$A = R S R^T, \quad R = \begin{pmatrix} \cos 45^\circ & -\sin 45^\circ \\ \sin 45^\circ & \cos 45^\circ \end{pmatrix}, \quad S = \begin{pmatrix} 3 & 0 \\ 0 & 0.5 \end{pmatrix}$$

読み方は**右から**（ベクトルに近い順）。モナ・リザで1段ずつ：



- ① R^T ： -45° 回して、「回らない方向」（赤・橙）を **座標軸にそろえる**
- ② S ：軸に沿って伸縮するだけ — ②の絵では赤い矢印が x 軸の上に乗っている
- ③ R ： 45° 回して、元の向きに戻す
- つまり「回らない方向」の正体は、 **S がただ伸縮するだけの軸を、 R が 45° に向けた先**
- **固有ベクトル = R の列**（軸の向き先）、**固有値 = S の対角成分**（3 と 0.5）

```
1 # 組み立てて確認：R S R^T がたしかに A になる
2 print(R_rot @ S_str @ R_rot.T)
```

Python

```
[[1.75 1.25]
 [1.25 1.75]]
```

💡 ヒント

対称行列は**かならず**この「回転・伸縮・逆回転」の形に分解できる — これが**固有値分解**。
`np.linalg.eigh` が返してくるのは、この R （固有ベクトル）と S の対角成分（固有値）。

答えが「固有ベクトル」になる理由

- $\|\mathbf{v}\| = 1$ は「 $\mathbf{v}$ の先端は半径1の球面上の上」という意味
- 球面上の **てっぺん** では、球面に沿ってどちらへ動いても $\mathbf{v}^T C \mathbf{v}$ は増えない
- 増える向きを表すのは勾配 $2C\mathbf{v}$ 。これが球面の法線方向（ $\mathbf{v}$ 自身）と **平行** になるしかない：

$C\mathbf{v} = \lambda\mathbf{v}$ （固有ベクトルの定義式そのもの！）

- しかもそのときの分散は $\mathbf{v}^T C \mathbf{v} = \lambda \mathbf{v}^T \mathbf{v} = \lambda$ — **固有値＝その方向の分散**

💡 ヒント

- **固有ベクトル**＝「球面上でこれ以上増やせない方向」の候補リスト
- **固有値**＝その候補での分散の値
- $\lambda_1 \geq \lambda_2 \geq \dots$ と並べて、一番大きい候補から PC1, PC2, ... と呼ぶ

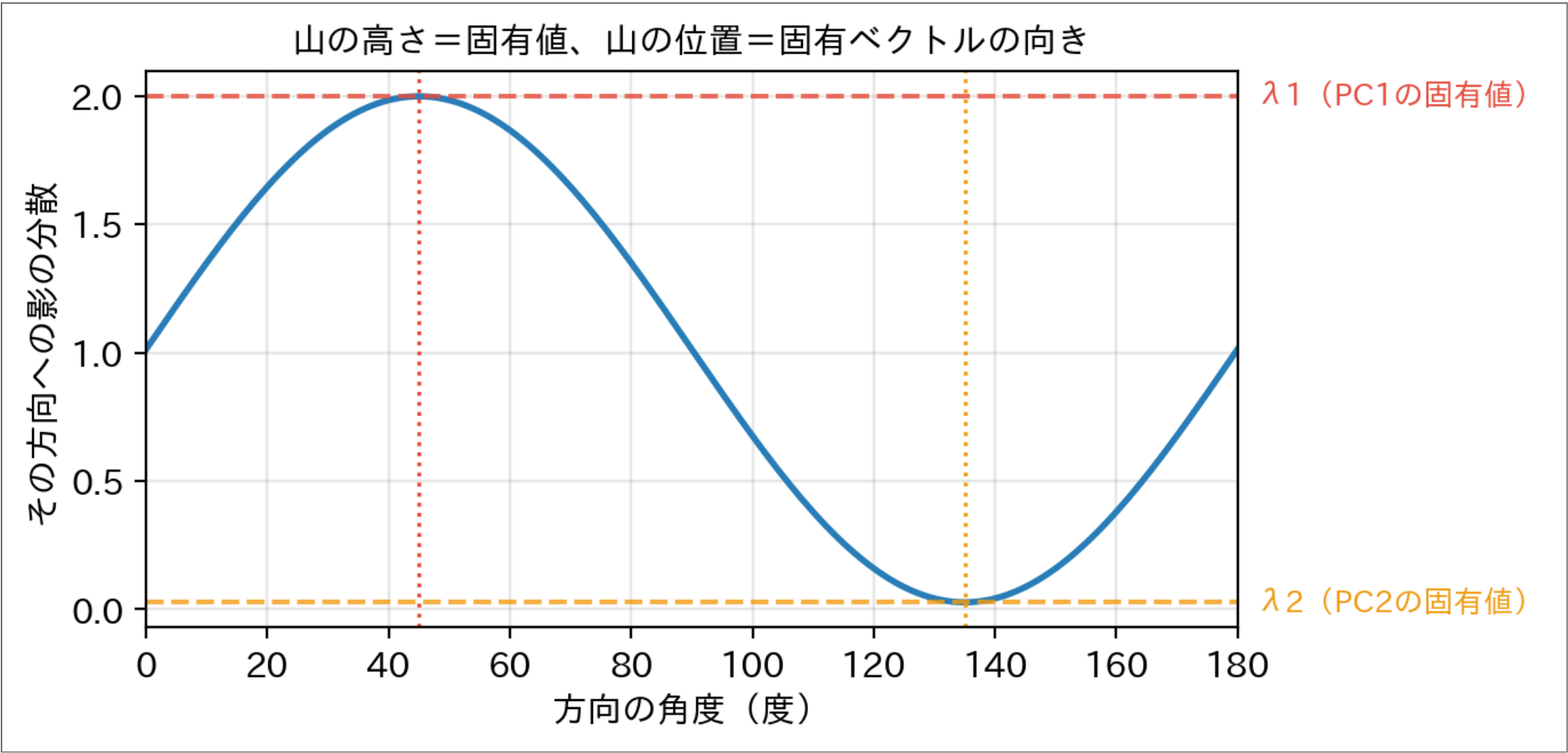
総当たりで確かめる：全方向の分散を測る

固有値分解を信じる前に、**力づく**で確認してみましょう。方向 $\mathbf{v}$ を1度刻みで全部試します。

Python

```
1 thetas = np.radians(np.arange(0, 181))
2 proj_var = np.array([
3     np.var(X_lw_sc @ np.array([np.cos(t), np.sin(t)]), ddof=1)
4     for t in thetas
5 ])
6
7 print(f"分散が最大の角度: {np.degrees(thetas[proj_var.argmax()]):.0f}°"
8       f" (分散 {proj_var.max():.4f}) ")
9 print(f"分散が最小の角度: {np.degrees(thetas[proj_var.argmin()]):.0f}°"
10       f" (分散 {proj_var.min():.4f}) ")
```

分散が最大の角度: 45° (分散 1.9978)
分散が最小の角度: 135° (分散 0.0275)



- 総当たりの最大値・最小値が、固有値 λ_1, λ_2 に**ぴったり一致**
- 山の位置（45°）が PC1 の向き、谷の位置（135°）が PC2 の向き
- この総当たりを**一発の計算**で済ませるのが固有値分解

NumPy で確かめる

スライドの式を **そのまま** コードにして、sklearn の答えと突き合わせます。

```
1 # 1) 式のとおりに関数を作る（専用関数はない）
2 Xc = X_lw_sc - X_lw_sc.mean(axis=0)      # 中心化 X~
3 C = Xc.T @ Xc / (len(Xc) - 1)           # C = X~^T X~ / (n-1)
4
5 # 2) 固有値・固有ベクトル（C のような対称行列には eig を使う）
6 lam, V = np.linalg.eigh(C)
7 order = np.argsort(lam)[::-1]           # 固有値の大きい順に並べ替え → PC1, PC2
8 lam, V = lam[order], V[:, order]
9
10 print("固有値（自作）          :", lam.round(4))
11 print("分散（sklearnのPCA）    :", pca_d.explained_variance_.round(4))
12 print("固有ベクトル（自作・列がPC）:")
13 print(V.round(4))
14 print("components_.T (sklearn) :")
15 print(pca_d.components_.T.round(4))
```

```
固有値（自作）          : [1.9978 0.0275]
分散（sklearnのPCA）    : [1.9978 0.0275]
固有ベクトル（自作・列がPC）:
[[ 0.7071 -0.7071]
 [ 0.7071  0.7071]]
components_.T (sklearn) :
[[ 0.7071  0.7071]
 [ 0.7071 -0.7071]]
```

- 固有値は sklearn の **explained_variance_** と **完全一致**
- ベクトルは列によって **符号が逆** になることがある — 軸の向きが180°逆なだけで、**同じ軸**

💡 実は sklearn の中身は SVD

実務の PCA は、共分散行列を作らずに中心化したデータを **SVD（特異値分解）** に かけて同じ答えを出している（大きいデータで速く、数値的にも安定）。

```
1 U, S, Vt = np.linalg.svd(Xc)
2 print("SVD から出した分散:", (S**2 / (len(Xc) - 1)).round(4))  # ↑の固有値と一致
```

```
SVD から出した分散: [1.9978 0.0275]
```


scikit-learn で PCA（魚は本当は何次元か）

- 輝度・重さ・体長の**3特徴量**を PCA にかけてみる

```
1 X3 = np.column_stack([brightness, weights, lengths])    # 3特徴量
2
3 X3_scaled = StandardScaler().fit_transform(X3)          # スケールを揃えてから PCA
4
5 pca = PCA(n_components=3)
6 X3_pca = pca.fit_transform(X3_scaled)
7
8 print("各主成分の寄与率（情報量の割合）：")
9 for i, r in enumerate(pca.explained_variance_ratio_):
10     print(f"  PC{i+1}: {r:.1%}")
```

各主成分の寄与率（情報量の割合）：
PC1: 79.2%
PC2: 19.9%
PC3: 0.9%

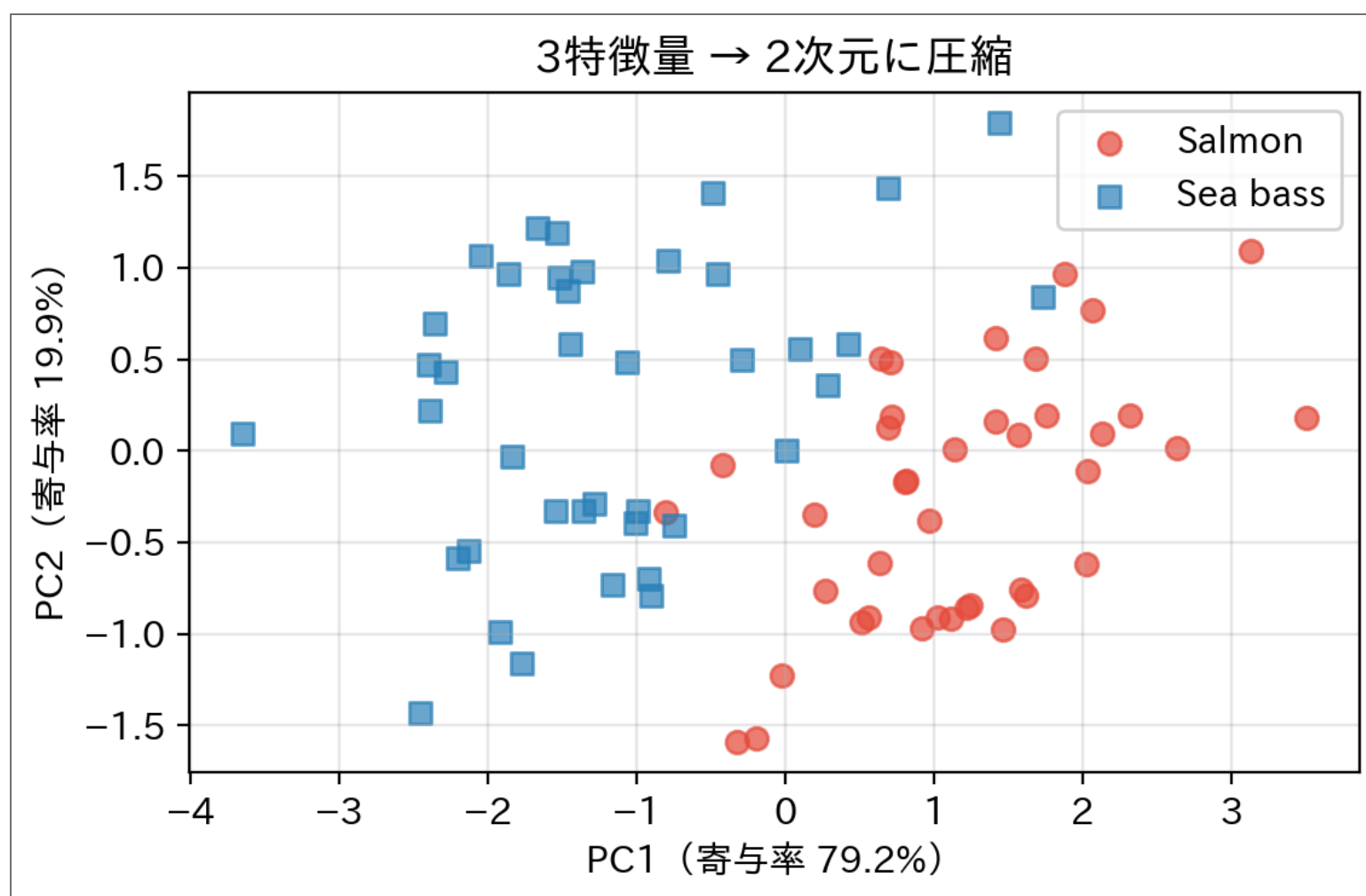
🚨 クイズ

PC3 の寄与率は **1% 未満**。これは何を意味している？

- 体長と重さがほぼ同じ情報だから、3本目の軸には**情報が残っていない**
- この魚データは、3次元に見えて**実質2次元**だった

2次元に圧縮しても何も失わない

```
1 colors = ["#e74c3c", "#2980b9"]
2
3 plt.figure(figsize=(5.8, 3.8))
4 for label, name, col, m in [(1, "Salmon", "#e74c3c", "o"), (0, "Sea bass", "#2980b9", "s")]:
5     mask = y == label
6     plt.scatter(X3_pca[mask, 0], X3_pca[mask, 1],
7                 c=col, marker=m, s=45, alpha=0.7, label=name)
8 plt.xlabel(f"PC1 (寄与率 {pca.explained_variance_ratio_[0]:.1%}) ")
9 plt.ylabel(f"PC2 (寄与率 {pca.explained_variance_ratio_[1]:.1%}) ")
10 plt.title("3特徴量 → 2次元に圧縮")
11 plt.legend(); plt.grid(True, alpha=0.3)
12 plt.tight_layout()
13 plt.show()
```



- 次元を1つ捨てたのに、2種類の魚は**今までどおり分離できている**
- 累積寄与率 99% — 捨てたのは「情報のない軸」だけ

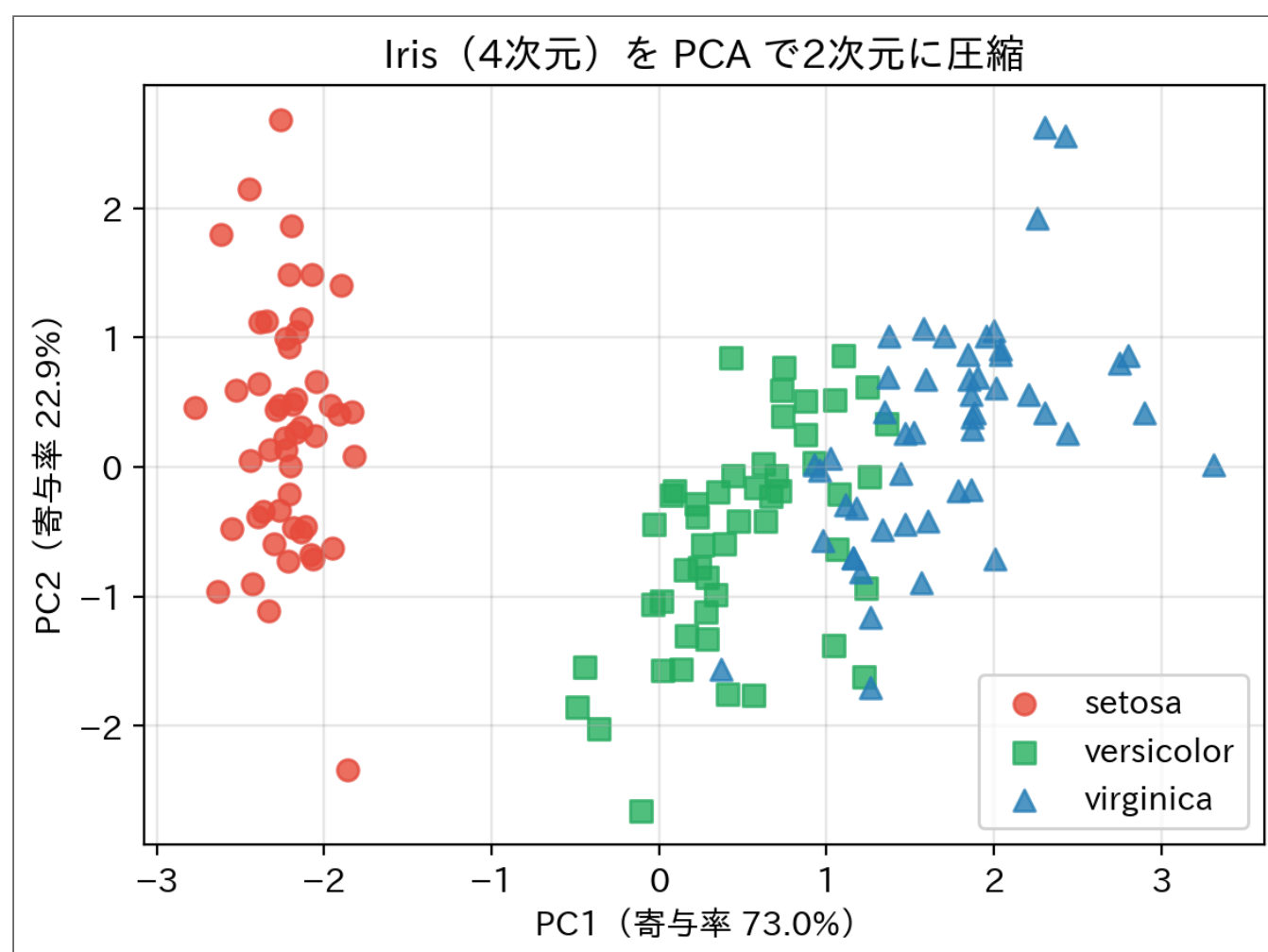
① StandardScaler を使う理由

輝度 (1~10) ・ 重さ (1000~2500g) ・ 体長 (45~62cm) はスケールが全く違う。正規化しないと重さだけが主成分を支配してしまう (クラスタリング回のスケールの罠と同じ)。

4次元データを2次元に圧縮（Iris）

Fisher の Iris データセット：4特徴量（がく片長さ・幅、花弁長さ・幅）

```
1 from sklearn.datasets import load_iris
2
3 iris = load_iris()
4 X_iris = StandardScaler().fit_transform(iris.data) # 4特徴量
5 y_iris = iris.target
6
7 pca4 = PCA(n_components=2)
8 X_2d = pca4.fit_transform(X_iris)
9
10 colors = ["#e74c3c", "#27ae60", "#2980b9"]
11 markers = ["o", "s", "^"]
12
13 plt.figure(figsize=(6, 4.5))
14 for label, name in enumerate(iris.target_names):
15     mask = y_iris == label
16     plt.scatter(X_2d[mask, 0], X_2d[mask, 1],
17                 c=colors[label], marker=markers[label],
18                 s=50, alpha=0.8, label=name)
19
20 plt.xlabel(f"PC1 (寄与率 {pca4.explained_variance_ratio_[0]:.1%}) ")
21 plt.ylabel(f"PC2 (寄与率 {pca4.explained_variance_ratio_[1]:.1%}) ")
22 plt.title("Iris (4次元) を PCA で2次元に圧縮")
23 plt.legend()
24 plt.grid(True, alpha=0.3)
25 plt.tight_layout()
26 plt.show()
```

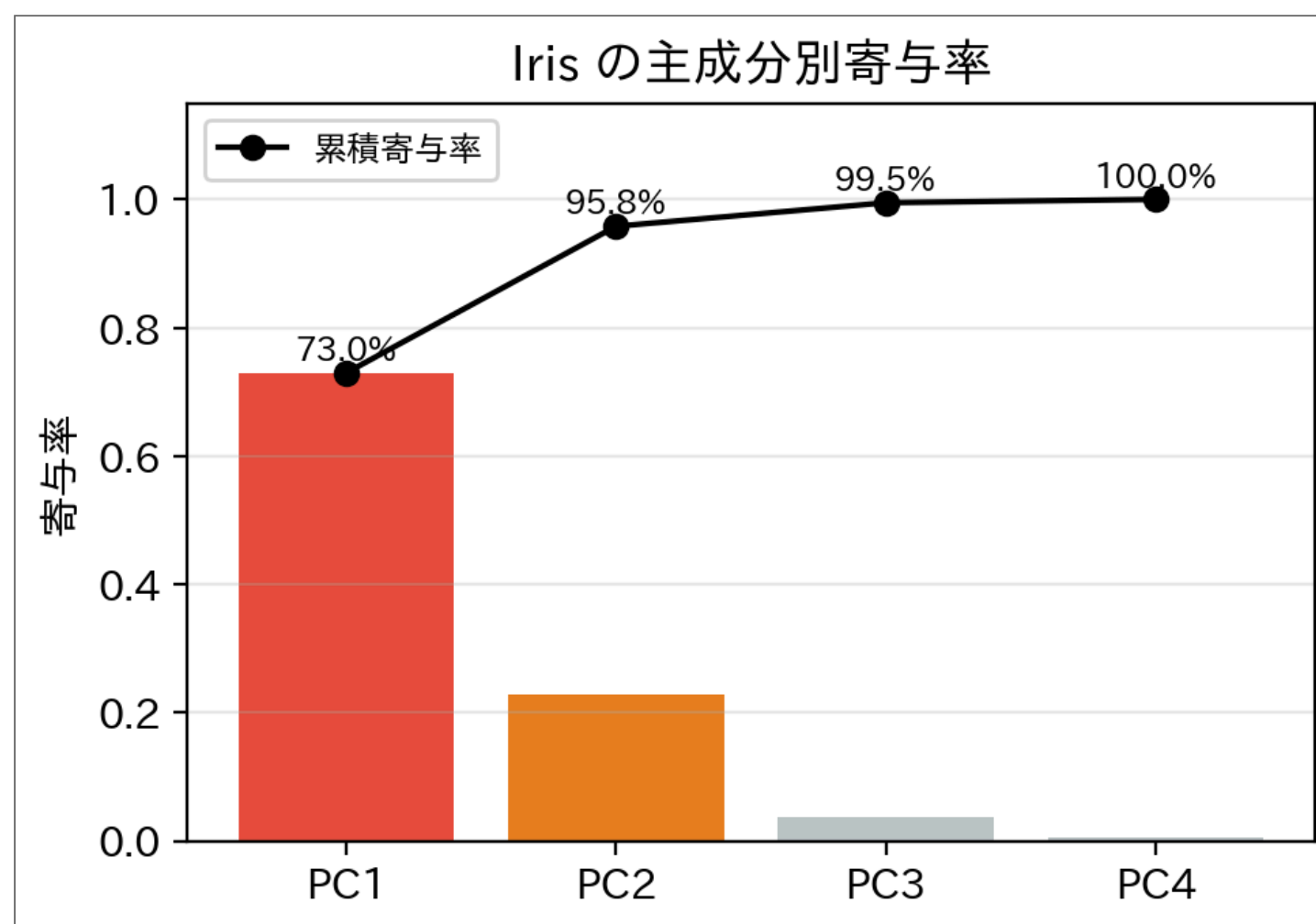


① ノート

4次元を2次元に圧縮しても全情報の 97% 以上が保たれている。 setosa（赤丸）が完全に分離できているのも図から見える。

寄与率 (Explained Variance Ratio)

```
1 ratios      = pca4.explained_variance_ratio_  
2 cumulative = np.cumsum(ratios)  
3  
4 fig, ax = plt.subplots(figsize=(5, 3.5))  
5 x_pos = ["PC1", "PC2", "PC3", "PC4"]  
6  
7 # 全4成分を改めて計算  
8 pca_full = PCA(n_components=4).fit(X_iris)  
9 ratios_full = pca_full.explained_variance_ratio_  
10 cum_full    = np.cumsum(ratios_full)  
11  
12 ax.bar(x_pos, ratios_full, color=["#e74c3c", "#e67e22", "#bdc3c7", "#bdc3c7"])  
13 ax.plot(x_pos, cum_full, "o-", color="black", label="累積寄与率")  
14 for x, y in zip(x_pos, cum_full):  
15     ax.text(x, y + 0.02, f"{y:.1%}", ha="center", fontsize=9)  
16 ax.set_ylabel("寄与率")  
17 ax.set_ylim(0, 1.15)  
18 ax.legend(fontsize=9)  
19 ax.set_title("Iris の主成分別寄与率")  
20 ax.grid(True, alpha=0.3, axis="y")  
21 plt.tight_layout()  
22 plt.show()
```



💡 何次元に削減するか？

一般的には**累積寄与率が 80~95%** を超える次元数を選ぶ。Iris は PC1+PC2 だけで 97% 超えるので2次元で十分。

まとめ

- **次元の呪い**：次元が増えると空間が広くなり、距離が意味をなさなくなる
 - **特徴量を増やしても情報が増えるとは限らない**：体長は重さの言い換えだった
 - **PCA**：分散が最大の方 向（＝ 共分散行列の固有ベクトル） に軸を取り直す
- 寄与率を見れば「データの **実質的な次元**」がわかる（魚は3次元に見えて実質2次元）

Python

```
1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 X_scaled = StandardScaler().fit_transform(X)    # スケール正規化
5 pca = PCA(n_components=2)
6 X_2d = pca.fit_transform(X_scaled)              # 2次元に圧縮
7
8 pca.explained_variance_ratio_    # 各 PC の寄与率
```

概念	意味
主成分（PC）	分散が最大の方 向（固有ベクトル）
固有値	その方 向の分散の大きさ
寄与率	情報量の割合（累積80～95%を目安に次元数を決める）

① 分類問題と特徴量のまとめ

1D → 2D の分類、評価と汎化、教師なし学習（k-means）、次元の呪いと PCA。 **機械学習の基本的な問いと道具が揃った。**

ここまで **model.fit(X, y)** という1行を何度も呼んだが、 **中身は一度も見えていない**。 次の「最適化」から、この1行を開けにいく。